보고서

휴머노이드 예비단원 이주원

1. 문제와 환경
   1. 과제 목표
   2. 관측
   3. 행동
   4. 종료와 초기화
   5. 보상 함수

2. 알고리즘과 코드 분석
   1. DQN과 PPO의 차이
   2. 실험 코드 구성
   3. 학습 진입과 경험 수집
   4. Return과 Advantage 계산
   5. 파라미터 갱신
   6. 행동 출력과 평가

3. 실험 설계
   1. 실험 조건
   2. 평가 지표
   3. 사전 가설

4. 결과와 고찰
   1. 학습 곡선
   2. 평가 결과 비교
   3. 보상 구성과 보행의 관계
   4. 영상 분석
   5. 한계와 후속 실험

1. 문제와 환경

1-1. 과제 목표

이 과제는 PPO 알고리즘으로 휴머노이드 로봇이 넘어지지 않고 앞으로 걸어가도록 학습시키는 것이다. 환경은 짐나시움 1.4.0의 Humanoid-v5이고, 물리 시뮬레이터는 무조코 3.17.0이다. PPO는 직접 구현하지 않고 SB3 2.9.2를 사용하였다. 대신 SB3 내부 코드를 따라가며 학습이 어떻게 진행되는지 분석하였다. 기본 설정과 제어 비용 가중치만 바꾼 설정을 같은 조건에서 학습시켜 비교하였다.

1-2. 관측

관측은 숫자 348개로 이루어져 있다. 환경 코드의 _get_obs가 6가지 정보를 차례로 이어 붙여 만든다.
첫번째는 몸통 높이와 방향, 각 관절의 각도 22개로, 로봇의 현재 자세이다. 원래는 24개인데 몸통의 x, y 위치 2개는 빠져 있다. 같은 자세라면 어디에 있든 같은 행동을 하도록 하기 위해서이다.
두번째는 몸통과 관절의 속도 23개이다. 자세만으로는 지금 어느 쪽으로 넘어지는 중인지 알 수 없기 때문에 속도가 필요하다.
나머지는 각 몸체의 질량 정보 130개, 각 몸체의 속도 78개, 관절 모터가 실제로 낸 힘 17개, 각 몸체가 바깥에서 받은 힘 78개이다. 마지막 바깥 힘으로 발이 땅에 닿았는지 알 수 있어서, 걷는 데 필요한 정보가 된다.

1-3. 행동

행동은 숫자 17개이고, 각각 관절 모터 하나에 주는 입력이다. 범위는 모두 -0.4~0.4이다. 실제 관절에 걸리는 힘은 이 입력에 gear 값을 곱한 크기이다.
복부 관절 3개는 gear가 100이다. 다리는 좌우 각각 4개씩 총 8개로, 엉덩이 좌우와 회전(100), 엉덩이 앞뒤(300), 무릎(200)이다. 팔은 좌우 각각 3개씩 총 6개로 모두 25이다. 다리를 앞뒤로 드는 엉덩이 관절과 무릎의 gear가 가장 크고, 팔은 다리의 1/12~1/8 정도이다. 같은 입력을 줘도 다리가 훨씬 큰 힘을 내므로, 몸을 받치고 앞으로 가는 일은 주로 다리가 맡게 된다.
카트폴은 왼쪽, 오른쪽 둘 중 하나만 고르면 되었지만, 휴머노이드는 매 스텝 17개 관절의 힘을 한꺼번에 정해야 한다.

1-4. 종료와 초기화

에피소드는 두 가지 경우에 끝난다. 첫번째는 몸통 높이가 1.0~2.0을 벗어난 경우로, 넘어진 것으로 보고 terminated가 된다. 시작할 때 몸통 높이는 1.4이다. 두번째는 1000스텝을 버틴 경우로, 시간 제한 때문에 truncated가 된다. 한 스텝은 0.015초이므로 1000스텝은 15초이다.
reset할 때는 시작 자세의 관절 위치와 속도에 -0.01~0.01 사이의 작은 무작위 값을 더한다. 매번 조금씩 다른 자세로 시작하게 해서 한 가지 자세에만 맞춰진 행동을 배우지 않게 하는 것이다. 같은 시드를 주면 같은 시작 자세가 나와 실험을 다시 할 수 있다. 학습 전에 무작위로 행동하게 해보니 평균 24스텝만에 넘어졌다.

1-5. 보상 함수

매 스텝의 보상은 자세 유지 보상 + 전진 보상 - 제어 비용 - 접촉 비용이다.

자세 유지 보상은 몸통 높이가 1.0~2.0 안에 있으면 5, 아니면 0이다. 넘어지는 마지막 스텝은 이미 범위를 벗어났으므로 0을 받는다.
전진 보상은 1.25 × 앞쪽 방향 속도이다. 이때 속도는 몸통이 아니라 몸 전체의 무게중심이 한 스텝 동안 움직인 거리로 계산한다. 몸통만 앞으로 숙여서는 무게중심이 덜 움직이므로, 몸 전체가 앞으로 가야 보상을 받는다.
제어 비용은 0.1 × 17개 입력을 각각 제곱해서 더한 값이다. 힘을 세게 줄수록 커진다. 모든 관절에 최대 입력 0.4를 줘도 0.1 × 17 × 0.16 = 0.272로, 자세 유지 보상 5에 비하면 작다.
접촉 비용은 바깥에서 받은 힘이 클수록 커지는데, 최대 10으로 제한되어 있다. 넘어지면서 땅에 세게 부딪혀도 비용이 끝없이 커지지 않게 막아둔 것이다.

2. 알고리즘과 코드 분석

2-1. DQN과 PPO의 차이

과제1 의 DQN은 행동마다 q값을 계산하고 그중 가장 큰 행동을 골랐다. 카트폴은 행동이 2개라 출력도 2개면 되었다. 하지만 휴머노이드의 행동은 소수 17개이다. 관절마다 -0.4, 0, 0.4 세 가지로만 나눠도 엄청나게 많은 조합이 생겨 q값을 다 계산할 수 없다. 그래서 q값을 비교하는 대신 행동을 바로 출력하는 방법이 필요하다.

PPO에는 신경망이 두 가지 있다. 액터는 17개 관절마다 어느 정도까지 힘을 주면 된다라는 평균값을 출력하고, 그 근처에서 무작위로 행동을 뽑는다. 크리틱은 지금 상태가 앞으로 얼마나 좋은지를 숫자 하나로 예상한다. dqn은 q네트워크 하나만 학습했지만, PPO는 둘 다 학습한다.
경험을 쓰는 방식도 다르다. DQN은 리플레이 버퍼에 경험을 2만 개까지 쌓아두고 예전 경험도 다시 꺼내 썼다. PPO는 지금 정책으로 2048스텝을 모아 그것으로 10번 학습한 뒤 버리고, 바뀐 정책으로 다시 모은다. PPO는 지금 정책으로 모은 경험으로만 지금 정책을 고치는 방식이라 예전 경험은 쓸 수 없다.

2-2. 실험 코드 구성

RLppo_hw.py는 학습과 평가를 한 파일에서 한다. 19~22줄에 기본 설정 base와 변경 설정 ctrl001을 정의하였고, 둘의 차이는 ctrl_cost_weight 하나뿐이다. 67~71줄에서 설정 이름, 시드, 학습 스텝 수, 평가 횟수를 명령어로 받기 때문에 코드를 고치지 않고 모든 조건을 실행할 수 있다.
85줄에서 환경을 만들고 Monitor로 감싼다. Monitor는 에피소드가 끝날 때마다 총 보상과 길이를 train.monitor.csv에 기록하고, 이 파일로 학습 곡선을 그린다. 89줄에서 SB3 기본값으로 PPO 모델을 만들고, 91줄에서 학습한다. 93줄에서 모델을 저장하고, 96줄에서 다시 불러와 평가한다.
26~62줄의 evaluate는 학습 환경과 별도로 평가용 환경을 만든다. 28줄에서 두 설정 모두 기본 보상으로 평가하게 하였다. 변경 설정은 학습할 때 보상 계산이 달랐기 때문에 같은 기준으로 비교하기 위해서이다. 32줄에서 평가 시드를 1000부터 주어 모든 모델이 같은 시작 자세로 평가받게 하였다.

2-3. 경험 수집

model.learn을 실행하면 SB3의 common/on_policy_algorithm.py 300줄 learn이 실행된다. learn은 정해진 스텝을 채울 때까지 collect_rollouts(324줄)로 경험을 모으고

self.train()(337줄)으로 학습하는 것을 반복한다.
collect_rollouts(162줄)는 지금 정책으로 2048스텝 동안 환경을 실행한다. 매 스텝 관측을 정책에 넣어 행동, 상태 가치, 그 행동을 뽑을 확률을 얻는다. 정책이 뽑은 행동은 범위 제한이 없으므로 216줄에서 환경에 넣기 직전에 -0.4~0.4로 자른다. 하지만 버퍼에는 자르기 전 행동을 저장한다(247줄). 나중에 학습할 때 그 행동을 뽑을 확률을 다시 계산해야 하는데, 잘린 값으로는 정확히 계산할 수 없기 때문이다.
234~245줄은 시간 제한으로 끝난 경우를 따로 처리한다. 1000스텝을 버텨 끝난 것은 넘어진 게 아니므로, 마지막 상태의 가치를 보상에 더해준다. hw1에서 버퍼에 terminated만 저장하고 truncated는 실패로 보지 않았던 것과 같은 이유이다.

2-4. 리턴과 어드밴티지 계산

경험을 다 모으면 common/buffers.py 403줄의 compute_returns_and_advantage가 마지막 스텝부터 거꾸로 계산한다.
먼저 433줄에서 delta = r + 0.99 × 다음 상태 가치 × (1 - 종료) - 현재 상태 가치를 구한다. 한 스텝 실제로 받은 결과가 크리틱의 예상과 얼마나 다른지를 나타낸다. hw1의 TD 오차와 같은 식이다. 434줄에서는 뒤쪽 스텝들의 delta를 조금씩 줄여가며 더해 어드밴티지를 구한다. 438줄에서 리턴 = 어드밴티지 + 현재 상태 가치로 계산한다.
리턴은 "이 상태에서 앞으로 받을 보상의 합"이고, 크리틱이 맞혀야 할 정답이 된다. 어드밴티지는 "이번 행동이 평소 예상보다 얼마나 좋았나"이다. 어드밴티지가 양수면 그 행동을 더 자주 하게, 음수면 덜 하게 정책을 고친다.

2-5. 파라미터 갱신

ppo/ppo.py 184줄의 train은 모은 2048개를 64개씩 나누어 10번 반복 학습한다. 213줄에서 지금 정책으로 각 행동을 뽑을 확률과 상태 가치를 다시 계산하고, 218줄에서 어드밴티지를 평균 0, 표준편차 1로 맞춘다.
222줄의 ratio는 새 정책이 그 행동을 고를 확률이 경험을 모을 때보다 몇 배인지를 나타낸다. 225~227줄의 정책 손실에서 ratio가 0.8~1.2를 벗어나면 그 이상은 이득이 없게 잘라버린다. 그래서 정책이 한 번에 크게 바뀌지 않는다. 이것이 PPO의 클리핑이다. 같은 경험으로 10번이나 학습하기 때문에, 이 제한이 없으면 정책이 경험을 모은 정책과 너무 달라질 수 있다. hw1에서 타겟 복사 주기를 100으로 줄였을 때 정답이 너무 자주 바뀌어 학습이 무너진 것처럼, 학습 대상이 급하게 바뀌는 것을 막는 장치이다.
244줄의 가치 손실은 크리틱의 예상과 리턴의 차이를 제곱한 평균으로, 크리틱이 리턴을 맞히도록 학습시킨다. 256줄에서 정책 손실과 0.5 × 가치 손실을 더한다. 탐험을 돕는 엔트로피 항도 있지만 기본값이 0이라 이번 실험에서는 쓰이지 않았다. 277줄에서 기울기 크기를 0.5로 제한하고 278줄에서 파라미터를 갱신한다. 학습률은 0.0003이다.

2-6. 행동 출력과 평가

액터와 크리틱은 각각 은닉층 64개, 64개를 갖고, 활성화 함수는 Tanh이다. 액터는 17개 관절의 평균값을 출력하고, 얼마나 넓게 흩어서 뽑을지(표준편차)는 따로 학습되는 값이다. 처음에는 표준편차가 1이다. 행동 범위가 -0.4~0.4인 것을 생각하면, 학습 초반에는 뽑힌 행동 대부분이 범위 끝에서 잘린다.
학습 중에는 평균 근처에서 무작위로 행동을 뽑아 탐험한다. 평가할 때는 deterministic=True로 무작위 없이 평균값을 그대로 행동으로 쓴다. hw1에서 학습

중에는 엡실론으로 탐험하고 평가할 때는 엡실론을 0으로 둔 것과 같은 역할이다.
전체 흐름을 정리하면 다음과 같다. learn이 시작되면 collect_rollouts가 지금 정책으로 2048스텝을 실행하며 행동, 상태 가치, 확률, 보상, 종료 여부를 버퍼에 저장한다. 다 모이면 compute_returns_and_advantage가 어드밴티지와 리턴을 계산한다. 그다음 train이 이 버퍼로 액터와 크리틱을 10번 반복 학습하고, 버퍼의 경험은 버린다. 다시 collect_rollouts로 돌아가 바뀐 정책으로 새 2048스텝을 모은다. 이것을 100만 스텝을 채울 때까지 약 489번 반복한다.

3. 실험 설계

3-1. 실험 조건

기본 설정 A는 SB3 PPO 기본값과 Humanoid-v5 기본값을 그대로 사용하였다. 변경 설정 B는 제어 비용 가중치만 0.1에서 0.01로 줄였고, 나머지 조건은 모두 같게 하였다.
학습은 조건마다 100만 스텝으로 정하였다. PPO는 2048스텝씩 모아서 학습하므로 실제로는 1,001,472스텝을 사용하였다. 학습 시드는 0과 1 두 개를 사용하여 한 조건당 두 번씩 처음부터 학습하였다. 관측 정규화는 사용하지 않았다.
평가는 학습이 끝난 모델을 다시 불러와 학습과 따로 진행하였다. 모델마다 20회 평가하였고, deterministic=True로 무작위 탐험 없이 행동하게 하였다. 평가 시드는 1000~1019로 모든 모델에 똑같이 주었다. 실행 환경은 Ubuntu, CPU, Python 3.12.3, PyTorch 2.15.0이고, 한 번 학습하는 데 543~604초가 걸렸다.

3-2. 평가 지표

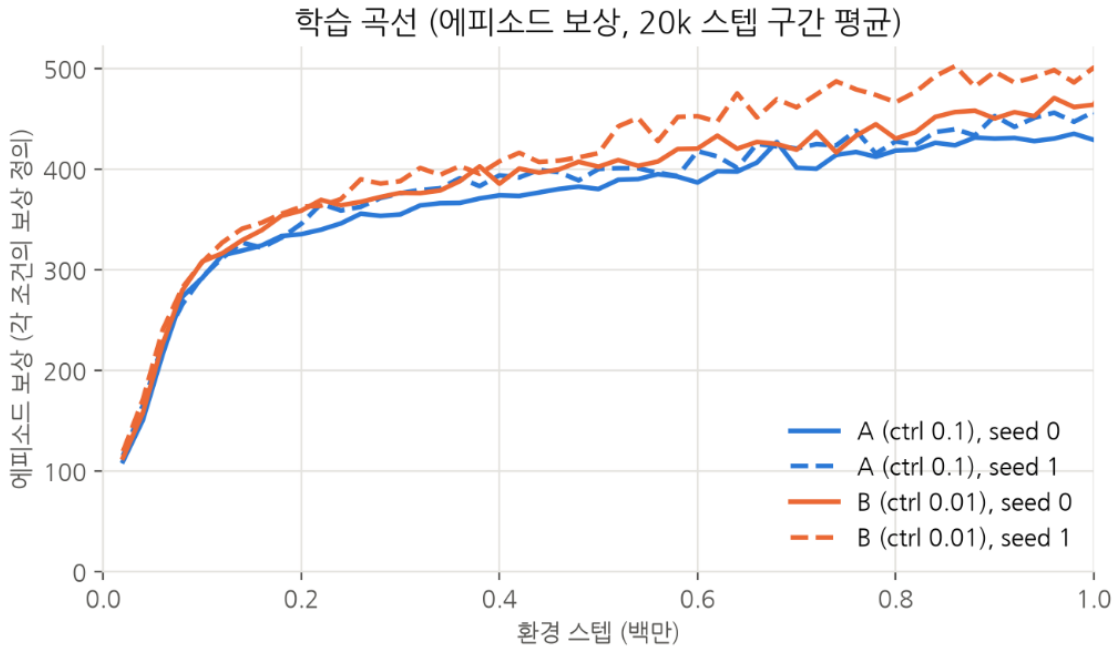
평가 보상은 두 조건 모두 기본 보상으로 계산한 에피소드 총합이다. B는 학습할 때 제어 비용을 덜 받았으므로, 학습 중 보상을 그대로 비교하면 B가 더 잘한 것처럼 보일 수 있다. 그래서 학습 곡선은 각 조건 안에서 어떻게 변하는지만 보고, 두 조건의 비교는 평가 보상과 아래 지표로 하였다.
유지 시간은 에피소드가 끝날 때까지 버틴 스텝 수이다. 넘어져서 끝났는지, 시간 제한으로 끝났는지도 같이 기록하였다.
평균 전진 속도는 무게중심이 앞으로 간 거리를 걸린 시간(스텝 수 × 0.015초)으로 나눈 값이다. 처음에는 몸통 위치로 계산했는데, 넘어질 때 몸통이 앞으로 쏠리는 거리까지 속도에 들어가는 문제가 있었다. 실제로 기본 설정 시드 0에서 몸통 기준으로는 0.67 m/s, 무게중심 기준으로는 0.53 m/s로 차이가 났다. 그래서 전진 보상과 같은 기준인 무게중심 속도를 사용하고, 몸통 속도는 참고로만 기록하였다.
제어 입력 크기는 평가하는 동안 모든 스텝, 모든 관절의 입력을 제곱해서 평균 낸 값이다. 최댓값은 0.4^2 = 0.16이다.

3-3. 사전 가설

이 가설은 B의 결과를 보기 전에 작성하였다. A 시드 0의 평가에서 제어 비용은 스텝당 평균 0.20으로, 스텝당 총 보상 5.25의 약 4%였다. 제어 입력 크기는 0.118로 최댓값의 74%였다.
B는 같은 힘을 써도 비용이 1/10로 줄어든다. 그래서 B는 힘을 아낄 이유가 줄어들어 제어 입력 크기가 커질 것으로 예상하였다. 하지만 A에서도 제어 비용은 전체 보상의 4%밖에 되지 않았으므로, 유지 시간과 평가 보상은 크게 다르지 않을 것으로 예상하였다. 이를 제어 입력 크기, 유지 시간과 넘어진 횟수, 무게중심 전진 속도, 평가 보상으로 확인하였다.

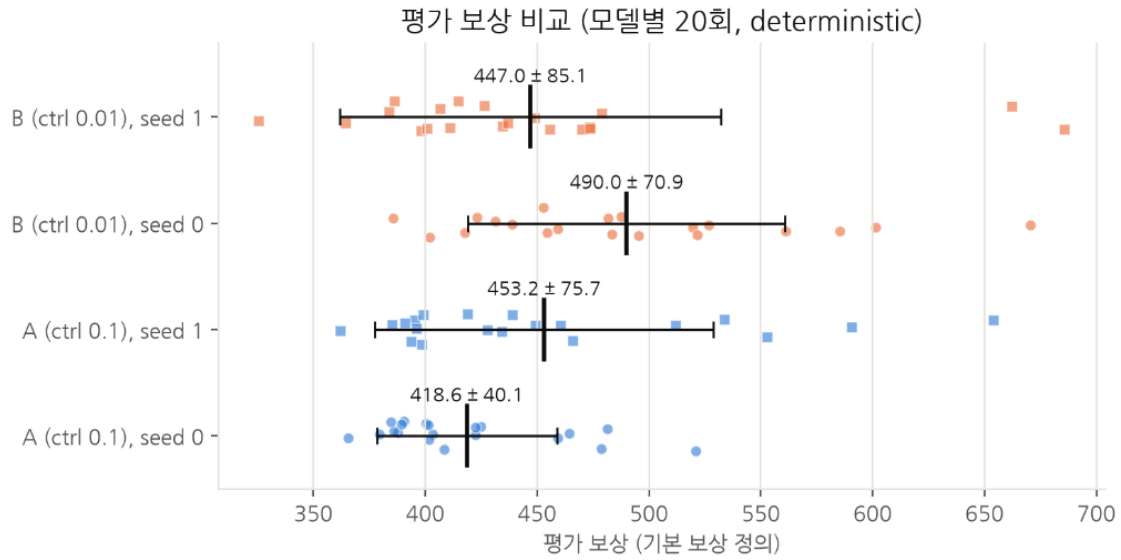
4. 결과 분석

4-1. 학습 곡선



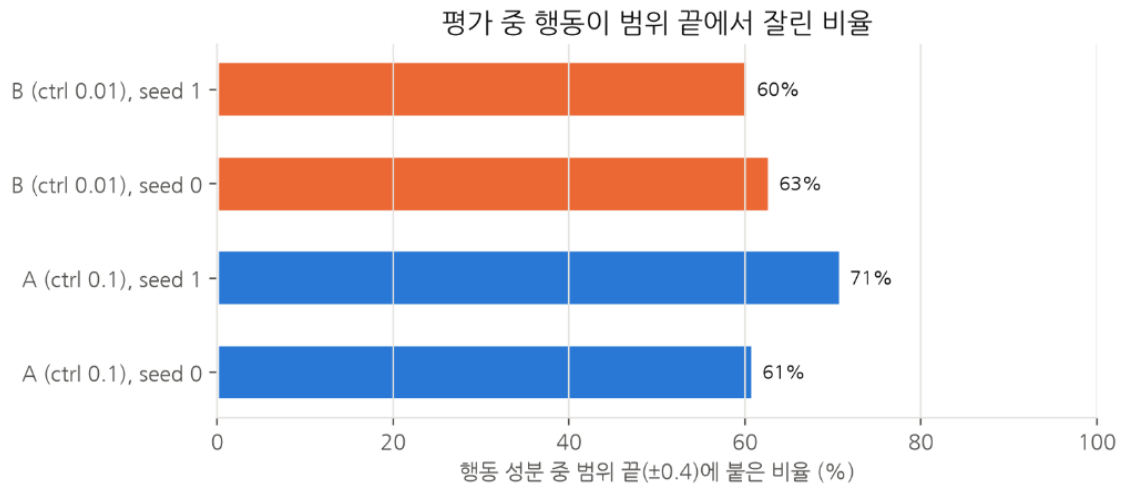
학습 곡선은 2만 스텝 구간마다 에피소드 보상과 버틴 스텝을 평균 내어 그렸다. 파란색이 A, 주황색이 B이고, 실선이 시드 0, 점선이 시드 1이다.
네 실행 모두 처음 20만 스텝 동안 거의 같은 모양으로 빠르게 올라갔다. A 시드 0은 처음 10만 스텝 동안 평균 보상 183(평균 40스텝 버팀)이었고, 40만~50만 스텝에서 379(78스텝), 마지막 10만 스텝에서 430(87스텝)이 되었다. A 시드 1은 191 > 395 > 450, B 시드 0은 191 > 401 > 461, B 시드 1은 200 > 412 > 492로, 네 실행 모두 비슷하게 올라갔다. 버틴 스텝 그래프도 20만 스텝까지 24에서 약 70으로 빠르게 늘고, 그 뒤로는 천천히 늘어 마지막에는 약 90스텝이 되었다.
최근 100 에피소드 평균이 가장 높았던 지점은 A 시드 0이 96만 스텝(452), A 시드 1이 95만 스텝(474), B 시드 0이 98만 스텝(494), B 시드 1이 86만 스텝(520)이었다. 네 실행 모두 학습이 거의 끝날 때 가장 높았다. 100만 스텝이 끝날 때까지 보상이 계속 오르고 있었으므로, 아직 다 배우지 못한 상태에서 학습이 멈춘 것이다.
학습 곡선의 보상은 각 조건의 보상으로 기록된 값이다. B는 제어 비용을 1/10만 받기 때문에, 같은 행동을 해도 스텝당 약 0.18, 에피소드당 약 16만큼 보상이 높게 나온다. B의 곡선이 A보다 조금 위에 있는 것은 이 차이 때문이기도 하므로, 학습 곡선으로는 두 조건을 비교하지 않았다. 실제로 학습 곡선에서 가장 높았던 B 시드 1(마지막 100 에피소드 평균 506.9)은 평가에서는 447.0으로 B 시드 0보다 낮았다.

4-2. 평가 결과 비교



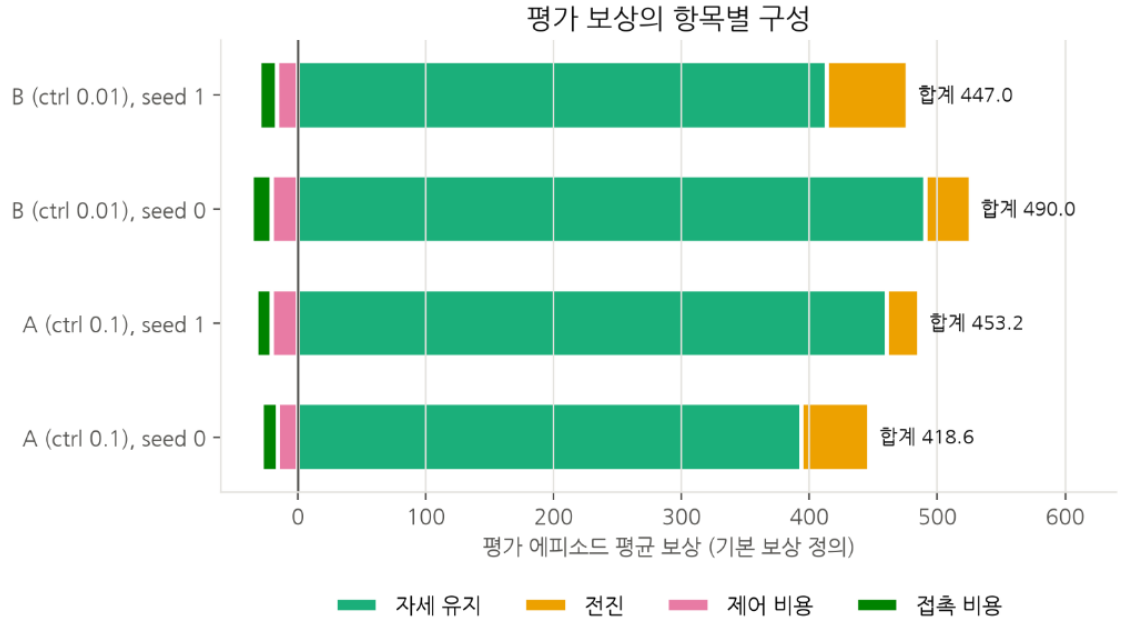
학습이 끝난 모델을 기본 보상 환경에서 20회씩 평가한 결과는 다음과 같다. 그림에서 점 하나가 평가 한 번이고, 검은 선은 평균과 표준편차이다.
평가 결과는 다음과 같다. A 시드 0은 평가 보상 418.6 ± 40.1, 버틴 스텝 79.8 ± 8.0, 무게중심 속도 0.53 ± 0.05 m/s, 제어 입력 크기 0.118이었다. A 시드 1은 453.2 ± 75.7, 93.1 ± 19.2스텝, 0.25 ± 0.18 m/s, 0.129였다. B 시드 0은 490.0 ± 70.9, 99.1 ± 15.0스텝, 0.30 ± 0.22 m/s, 0.121이었다. B 시드 1은 447.0 ± 85.1, 83.8 ± 15.4스텝, 0.60 ± 0.05 m/s, 0.116이었다. 네 모델 모두 20회 중 20회 넘어져서 끝났다.

네 모델 모두 20회 평가에서 한 번도 1000스텝을 버티지 못하고 넘어졌다. 가장 오래 버틴 경우도 145스텝(약 2.2초)이었다. 무작위로 움직일 때의 24스텝보다는 3~4배 오래 버티지만, 걷는다고 할 수준은 아니었다.
조건별로 두 시드를 평균하면 A는 평가 보상 435.9, 버틴 스텝 86.5, 속도 0.39 m/s, 제어 입력 크기 0.124였다. B는 468.5, 91.5스텝, 0.45 m/s, 0.119였다. B가 평가 보상이 32.6 높고 5스텝 더 버텼다. 하지만 같은 조건 안에서 시드만 바꿔도 평가 보상이 A는 34.6, B는 43.0만큼 달라졌고, 버틴 스텝도 A는 13.3, B는 15.3스텝 차이가 났다. 설정을 바꿔서 생긴 차이가 시드를 바꿔서 생긴 차이보다 작으므로, 이 실험만으로는 B가 A보다 낫다고 말할 수 없다.
B 시드 1은 같은 명령으로 한 번 더 학습했는데, 평가 결과가 소수점까지 똑같았다(447.0 ± 85.1). 같은 시드와 같은 설정이면 같은 결과가 나오므로, 이 결과는 다시 실행해도 재현할 수 있다.
가설 검증: 첫번째 가설인 “B는 제어 입력이 커질 것이다”는 틀렸다. B의 제어 입력 크기는 0.116~0.121로, A의 0.118~0.129와 거의 같거나 오히려 조금 작았다. 기본 보상으로 계산한 스텝당 제어 비용도 A가 0.20, 0.22, B가 0.21, 0.20으로 비슷했다. 두번째 가설인 "버틴 시간과 평가 보상은 크게 다르지 않을 것이다"는 맞았다.

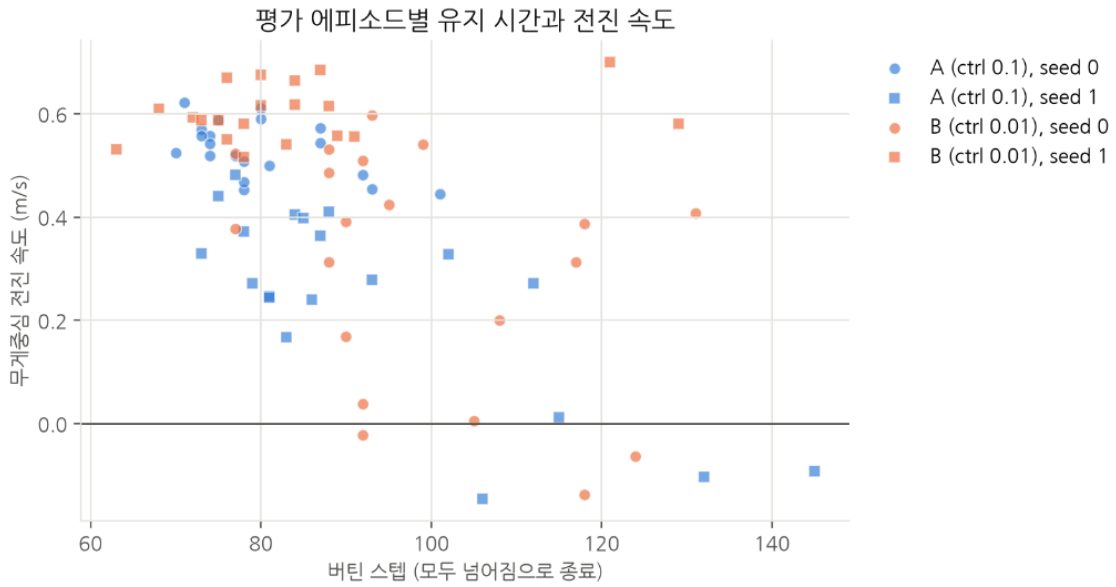


제어 입력이 커지지 않은 이유를 알아보기 위해, 평가 중 행동이 범위 끝(±0.4)에 붙어 있는 비율을 재보았다. 네 모델 모두 전체 입력의 60~71%가 ±0.4에 붙어 있었다. 정책이 출력한 평균값의 크기가 0.68~0.76으로, 이미 행동 범위 0.4를 넘어서 잘린 값이 들어가고 있었다. 이렇게 입력이 이미 끝에 붙어 있으면, 제어 비용 때문에 정책이 출력을 조금 줄여도 잘린 뒤의 실제 입력은 0.4 그대로이다. 그래서 제어 비용 가중치를 바꿔도 실제 제어 입력은 달라지지 않은 것으로 보인다.
또 학습된 정책의 표준편차는 0.71~0.75로, 행동 범위의 절반 폭인 0.4보다 컸다. 2-6에서 본 것처럼 SB3는 표준편차 1로 시작하는데, 행동 범위가 -0.4~0.4로 좁아서 학습하는 동안에도 뽑힌 행동의 상당수가 잘린 것으로 보인다. 결국 학습된 정책은 힘을 조절하기보다 관절을 끝까지 밀거나 당기는, 켜고 끄는 식의 제어에 가깝다.

4-3. 보상 구성과 보행의 관계



평가 보상을 항목별로 나누면, 네 모델 모두 자세 유지 보상이 평가 보상과 거의 같은 크기였다.
A 시드 1과 B 시드 0은 자세 유지 보상이 평가 보상보다도 크다. 전진 보상으로 얻은 것보다 제어 비용과 접촉 비용으로 잃은 것이 더 많았기 때문이다. 즉 이 환경의 점수는 대부분 앞으로 간 거리가 아니라 서 있었던 시간에서 나온다.



네 모델은 설정과 상관없이 두 가지 행동으로 나뉘었다.
A 시드 0과 B 시드 1은 무게중심 속도가 0.53, 0.60 m/s로 빠르고, 평가할 때마다 속도가 비슷했다(표준편차 0.05). 대신 79.8, 83.8스텝 만에 넘어졌다. 또 몸통 속도(0.67, 0.75 m/s)가 무게중심 속도보다 커서, 몸통이 몸 전체보다 앞으로 더 쏠렸다. 앞으로 기울어지면서 넘어지는 동작으로 보인다.
반대로 A 시드 1과 B 시드 0은 속도가 0.25, 0.30 m/s로 느렸지만 93.1, 99.1스텝으로 더 오래 버텼다. 몸통 속도와 무게중심 속도도 0.27로 거의 같았다. 이 두 모델은 평가할 때마다 속도 차이가 컸고(표준편차 0.18, 0.22), 각각 20회 중 3회는 속도가 0보다 작았다. 앞으로 가지 않고 제자리에서 버티거나 뒤로 밀려난 경우도 있었다는 뜻이다. 그림에서도 빠른 두 모델의 점은 왼쪽 위(짧고 빠름)에 모여 있고, 느린 두 모델의 점은 오른쪽 아래(길고 느림)까지 넓게 퍼져 있다.
평가 보상은 느리지만 오래 버틴 두 모델(453.2, 490.0)이 빠르게 앞으로 간 두 모델(418.6, 447.0)보다 높았다. 전진 보상은 1.25 × 속도라서 0.6 m/s로 가도 스텝당 0.75밖에 안 된다. 반면 서 있기만 해도 스텝당 5를 받는다. 그래서 이 보상에서는 앞으로 빨리 가는 것보다 조금이라도 오래 서 있는 쪽이 점수가 높다. 총 보상이 높다고 해서 잘 걷는다는 뜻이 아니고, 이번 결과에서는 오히려 더 많이 앞으로 간 모델의 점수가 더 낮았다.
어떤 행동을 배울지는 제어 비용 가중치보다 시드에 따라 정해졌다. 다만 모델이 4개뿐이라 이런 경향이 보인다는 정도이고, 이것만으로 확실하다고 할 수는 없다.

4-4. 영상 분석

영상을 보면 A seed 0은 앞으로 걸으려는 동작을 하다가 넘어졌다. 4-3에서 무게중심 속도가 0.53 m/s로 빠르고, 몸통이 몸 전체보다 앞으로 더 쏠렸던 것과 맞는 모습이다. 반면 A seed 1은 앞으로 나아가지 않고 거의 제자리에 서 있다가 넘어졌다. 무게중심 속도가 0.25 m/s로 느리고, 20회 중 3회는 뒤로 움직였던 것과 맞는다. 두 모델은 같은 설정으로 학습했지만 seed에 따라 앞으로 가려는 행동과 제자리에서 버티는 행동으로 갈렸다. 평가 보상은 제자리에서 버틴 seed 1(453.2)이 앞으로 가려던 seed 0(418.6)보다 높았다. 이 보상 구조에서는 앞으로 가는 것보다 오래 서 있는 것이 점수에 더 유리하다는 것을 영상으로도 확인할 수 있다.

B seed 0은 영상에서 조금 걷는 모습을 보였다. 다만 영상으로 고른 두 에피소드의 무게중심 속도는 0.42, 0.52 m/s로, 이 모델의 20회 평균 0.30 m/s보다 빨랐다. 이 모델은 20회 중 3회가 뒤로 움직였을 만큼 에피소드마다 차이가 크므로(표준편차 0.22), 영상의 걷는 모습이 모든 에피소드를 대표한다고 보기는 어렵다.

4-5. 한계와 후속 실험

첫번째 한계는 학습 시드가 조건마다 2개뿐이라는 점이다. 4-2에서 본 것처럼 시드에 따른 차이가 설정에 따른 차이보다 커서, 설정이 얼마나 영향을 주는지 구분하기 어려웠다. 두번째는 학습 스텝이다. 네 실행 모두 학습 곡선이 끝까지 오르고 있었으므로, 100만 스텝은 이 환경을 학습시키기에 부족했다. 세번째는 관측 정규화를 쓰지 않은 점이다. 관측 348개는 값의 크기가 서로 크게 다른데 그대로 신경망에 넣었으므로, 이것도 학습을 느리게 했을 수 있다.

후속 실험으로는 먼저 행동 범위 문제를 확인해야 한다. 행동을 -1~1 범위로 바꿔주는 RescaleAction을 쓰거나 표준편차 초깃값을 낮추고, 나머지 조건은 그대로 두고 비교한다. 이때 행동이 범위 끝에 붙는 비율이 줄어드는지, 버틴 시간이 늘어나는지를 확인한다. 이 문제를 해결한 뒤에 제어 비용 가중치 실험을 다시 해야 가중치의 효과를 제대로 볼 수 있다. 또 시드를 5개 이상으로 늘리고 학습 스텝도 늘려서, 시드 때문에 생긴 차이와 설정 때문에 생긴 차이를 구분해야 한다.